

Machine Learning (UFCFAS-15-2)

Week 1 · Day 2 Lab Task Guide: Hands-On Containerized ML & Docker Hub

Level 5

UFCFAS-15-2

15 UK Credits / 7.5 ECTS

Lecturer: Sudan Pudasaini

Duration: 90 Mins

UFCFAS-15-2 Machine Learning · Week 1 · Day 2 Lab Task Guide

Hands-On Containerized ML: Build, Train, Push & Pull with Docker

Module: Machine Learning (Level 5 · 15 UK Credits / 7.5 ECTS · UFCFAS-15-2)

Institution: The British College Nepal · Partnered with UWE Bristol

Lecturer: Sudan Pudasaini

Session: Class 2 of 3 (1.5 Hours / 90 Minutes)

Lab Objectives

By the end of today's 90-minute hands-on workshop, you will:

- 1. Understand Containerization for Machine Learning:** See why ML engineering relies on Docker to prevent dependency conflicts and "works on my machine" issues.
- 2. Build Your First Containerized ML Model:** Author a clean `Dockerfile` packaging a Python script, a training dataset (`data.csv`), and `scikit-learn` 's `LinearRegression` .
- 3. Execute Training Inside the Container:** Run the containerized training pipeline and evaluate model metrics (R^2 , MSE, slope, intercept).
- 4. Persist Trained Model Artifacts (Volume Mounting):** Mount a host folder (`-v $(pwd)/output:/app/output`) to save the trained model artifact (`model.joblib`) onto your laptop.
- 5. Publish to Docker Hub:** Tag and push your customized ML image to your public Docker Hub account (`<username>/my-first-ml:v1`).
- 6. Pull the Module Toolkit:** Pull the lecturer's official course image (`docker pull alkimi/ml:week1`) and test JupyterLab for Class 3.

TIP

****Why ML Engineers Need Docker:**** If you try running `python3 train_and_predict.py` directly on your host laptop without setup, it will fail with `ModuleNotFoundError: No module named 'numpy'`. Docker encapsulates Python, NumPy, Pandas, Scikit-Learn, and OS dependencies into an identical, reproducible sandbox across Mac, Windows, and Linux.

90–Minute Workshop Timeline

```
gantt
    title Week 1 · Day 2 Workshop Flow (90 Mins)
    dateFormat mm
    axisFormat %M min
    section Orientation (15m)
    Why Docker for ML & Smoke Test           :00, 15m
    section Hands-On ML Container (35m)
    Write ML Script, Data & Dockerfile       :15, 20m
    Build & Run Local ML Container           :35, 15m
    section Persistence & Cloud (25m)
    Volume Mounting (Persist Model)         :50, 10m
    Push to Personal Docker Hub              :60, 15m
    section Course Toolkit & Wrap (15m)
    Pull alkimi/ml:week1 & Launch Lab        :75, 10m
    Verification & Day 3 Preview             :85, 05m
```

Task 1: Docker Smoke Test & Verification (15 Mins)

1. Ensure **Docker Desktop** is running:
 - **Mac:** Look for the Docker whale icon in your top menu bar.
 - **Windows:** Ensure Docker Desktop is active (WSL2 backend running).
2. Open your terminal (macOS Terminal / Windows PowerShell) and check your Docker version:
`bash docker --version`
Expected output: `Docker version 24.x.x` (or newer).
3. Run the standard Docker hello-world test:
`bash docker run hello-world`
4. **Observe what happened:**
 - Docker checked your laptop locally for `hello-world`.
 - Since it wasn't found, it pulled the image layers from **Docker Hub**, launched an isolated container, executed the message, and exited.

Task 2: Build Your Containerized ML Model (35 Mins)

You are going to build a self-contained Machine Learning training application that predicts exam scores based on study hours.

Step 2.1: Create Project Workspace

In your terminal, create a new directory for today's lab:

```
mkdir -p ~/ml-labs/week1/docker-ml
cd ~/ml-labs/week1/docker-ml
```

Step 2.2: Create the Training Dataset (`data.csv`)

Create a file named `data.csv` containing student study hours vs exam scores:

```
study_hours,exam_score
1.5,42.0
2.0,50.0
2.5,53.5
3.0,58.0
3.5,62.0
4.0,66.5
4.5,70.0
5.0,74.5
5.5,78.0
6.0,81.5
6.5,85.0
7.0,89.0
7.5,92.5
8.0,95.0
9.0,98.5
```

Step 2.3: Create the Python ML Script (`train_and_predict.py`)

Create a file named `train_and_predict.py` :

```

import os
import sys
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import joblib

def main():
    print("=" * 60)
    print("🚀 UFCFAS-15-2 Machine Learning: Containerized Model Training")
    print("=" * 60)

    data_file = "data.csv"
    if not os.path.exists(data_file):
        print(f"❌ Error: Data file '{data_file}' not found.")
        sys.exit(1)

    print(f"📊 Loading dataset from '{data_file}'...")
    df = pd.read_csv(data_file)
    print(f"    Dataset shape: {df.shape[0]} rows, {df.shape[1]} columns")
    print(f"\nSample records:")
    print(df.head(5).to_string(index=False))

    # Feature matrix X and target vector y
    X = df[["study_hours"]].values
    y = df["exam_score"].values

    print(f"\n🧠 Training Linear Regression model...")
    model = LinearRegression()
    model.fit(X, y)

    slope = model.coef_[0]
    intercept = model.intercept_
    y_pred = model.predict(X)

    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)

    print(f"\n📈 Model Training Complete!")
    print(f"    Learned Equation: Exam Score = ({slope:.2f} × Study Hours) + {intercept:.2f}")
    print(f"    Slope (Weight w):      {slope:.4f}")
    print(f"    Intercept (Bias b):    {intercept:.4f}")
    print(f"    R² Score (Accuracy):  {r2:.4f} ({r2 * 100:.1f}%)")
    print(f"    Mean Squared Error:   {mse:.4f}")

    # Run test predictions
    print(f"\n🧪 Inference Test (New Predictions):")
    test_hours = np.array([[3.0], [5.0], [7.5], [10.0]])
    predictions = model.predict(test_hours)
    for hours, pred in zip(test_hours.ravel(), predictions):
        print(f"    Studying {hours:4.1f} hours/week ➡ Predicted Score: {pred:5.1f} / 100")

    # Save model artifact
    output_dir = "output"
    os.makedirs(output_dir, exist_ok=True)
    model_path = os.path.join(output_dir, "model.joblib")

```

```

joblib.dump(model, model_path)
print(f"\n📁 Model successfully saved to: '{model_path}')"
print("=" * 60)
print("✅ Container run successfully finished!")
print("=" * 60)

if __name__ == "__main__":
    main()

```

Step 2.4: Specify Dependencies (`requirements.txt`)

Create a file named `requirements.txt` :

```

numpy ≥ 1.26.0
pandas ≥ 2.2.0
scikit-learn ≥ 1.4.0
joblib ≥ 1.3.0

```

Step 2.5: Author the `Dockerfile`

Create a file named `Dockerfile` (capital `D`, no extension):

```

# 1. Official lightweight Python image
FROM python:3.11-slim

# 2. Set working directory inside container
WORKDIR /app

# 3. Copy dependencies and install without caching wheels
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. Copy training dataset and Python ML application
COPY data.csv .
COPY train_and_predict.py .

# 5. Create directory for output artifacts
RUN mkdir -p /app/output

# 6. Default execution: run the ML training & inference pipeline
CMD ["python", "train_and_predict.py"]

```

Step 2.6: Build the Docker Image

In `~/ml-labs/week1/docker-ml` , build your image:

```
docker build -t my-first-ml:v1 .
```

(Notice the trailing dot `.` – this tells Docker to build from the current directory).

Step 2.7: Verify the Built Image

```
docker images
```

You should see `my-first-ml` listed with tag `v1` .

Task 3: Run the Container & Mount a Volume (15 Mins)

Step 3.1: Run the Container Directly

Execute the container and watch it train:

```
docker run --rm my-first-ml:v1
```

Notice: The `--rm` flag cleans up the container process once the training completes.

Step 3.2: Persist the Trained Model to Your Laptop (Volume Mount)

By default, files written inside a container disappear when the container stops. To extract our trained model artifact (`model.joblib`), we mount a host directory:

```
mkdir -p output
docker run --rm -v $(pwd)/output:/app/output my-first-ml:v1
```

Now list files in your local `output` directory:

```
ls -la output/
```

Expected: You will see `model.joblib` directly on your laptop filesystem!

Task 4: Push to Your Docker Hub Account (15 Mins)

Just like in the CCD course, you will publish your container to **Docker Hub** so that it can be shared, cited in your portfolio, and deployed in the cloud.

Step 4.1: Log into Docker Hub

If you don't already have a Docker Hub account, create one at hub.docker.com.

Log in from your terminal:

```
docker login
```

Enter your Docker Hub username and password (or personal access token).

Step 4.2: Tag Your Image with Your Docker Hub Username

Docker Hub requires images to be tagged as `<username>/<repository>:<tag>`:

```
# Replace <YOUR_DOCKERHUB_USERNAME> with your actual Docker Hub username:
docker tag my-first-ml:v1 <YOUR_DOCKERHUB_USERNAME>/my-first-ml:v1
```

Example:

```
docker tag my-first-ml:v1 studentjohn/my-first-ml:v1
```

Step 4.3: Push to Docker Hub

```
docker push <YOUR_DOCKERHUB_USERNAME>/my-first-ml:v1
```

Step 4.4: Verify Online

Open your browser and navigate to:

👉 https://hub.docker.com/r/<YOUR_DOCKERHUB_USERNAME>/my-first-ml

You will see your public ML container live on the internet!

🎓 Task 5: Pull Lecturer's Pre-Built Course Image (`alkimi/ml:week1`) (10 Mins)

Now pull the official course toolkit provided by your lecturer:

Step 5.1: Pull the Official Week 1 Image

```
docker pull alkimi/ml:week1
```

Step 5.2: Launch the Course JupyterLab Environment

Run the official image with port forwarding and volume mounting:

```
docker run -p 8888:8888 -v $(pwd):/home/student/notebooks/mywork alkimi/ml:week1
```

Step 5.3: Verify in Your Browser

Open: <http://localhost:8888>

You should see JupyterLab running with:

- Python 3.12
 - Scikit-Learn, Pandas, NumPy, Matplotlib, Seaborn pre-configured
 - Zero local installation overhead!
-

✅ Task 6: Submission & Reflection (5 Mins)

In your course GitHub repository under the Course Organization, create a short

`WEEK1_DAY2_VERIFICATION.md` :

```
# Week 1 · Day 2: Docker ML Lab Verification

**Student Name:** [Your Full Name]
**Student ID:** [Your Student ID]
**Docker Hub Repository:** https://hub.docker.com/r/<YOUR_DOCKERHUB_USERNAME>/my-first-ml

## 1. Local Training Output
- Learned Equation: Exam Score = (7.50 × Study Hours) + 35.00
- R2 Score: 0.9850

## 2. Verification Screenshot / Terminal Proof
- Pushed custom ML image to Docker Hub
- Pulled official course image `alkimi/ml:week1`
- Launched JupyterLab on localhost:8888 successfully
```

Looking Forward to Class 3 (Tomorrow — 1 Hour)

Tomorrow in Class 3, we dive into the mathematics and code behind what we ran today:

- How does `LinearRegression()` find the optimal slope and intercept?
- The Loss Function: Mean Squared Error (MSE).
- Optimization via Gradient Descent: Walking down the error surface.
- Hands-on interactive Jupyter notebook inside `alkimi/ml:week1` !